Counter evaluation

A smaller equality interface for intuitionistic primitive recursive arithmetic

Research report prepared for Vladimir Reshetnikov

November 11, 2026

Repository snapshot: `831081033253cca2115196cf86bb99c77ed2aeae`

Abstract

On the repository's existing counter-aware, successor-seeded recursor, we replace separate computation and equality-transport mechanisms by reflexivity and one uniform binary rule. From $s = t$ and $R(Ss, a; i, u.h) = r$, the rule infers $r = h(t, R(t, a; i, u.h))$. Two applications recover root equality composition; two applications also recover congruence in any fixed term context. Neither derivation uses induction, numerical cases, or a logical rule.

Together with the existing Eq/D clause interface, this gives a complete five-pattern presentation bi-interpretable with full first-order iPRA. Both interpretation domains are whole and both numerical composites have isomorphism graph $x = y$. The paper specifies every rule, accounts for zero-body normalization, proves the interpretations and their composites, and supplies standalone checkers and executable proof transformations.

The complete new checker has 3004 normalized bytes, 854 significant tokens, and 987 AST nodes. The matched computation-plus-transport control has 3272 bytes, 929 tokens, and 1063 nodes. This is a local mathematical and implementation improvement, not a global minimum: a raw two-case scalar control is five bytes smaller, and the historical 516-byte compact encoding is not surpassed. The recursor, its direct primitive-recursion compiler, clause logic, and canonical zero convention are attributed prior work. The new contribution is their uniform counter-evaluation interface and its equality reconstruction.

Contents

1 What is simpler, and what is inherited 3

2 Numerical syntax and canonical substitution 3

3 The complete five-pattern calculus 4

4 Equality and computation from one evaluation rule 5

5 The logical and induction interface is complete 6

6 The inherited arithmetic bridge in a short form 9

7 Why equation induction gives all quantifier-free induction 10

8 The two interpretations and their round trips 11

9 Executable artifacts and verification boundary 12

10 Further research questions 14

A Novelty audit and attribution 14

B Complete standalone checker 15

1 What is simpler, and what is inherited

The supplied repository is unusually far along: its main index describes fifty research packages, a Lean formalization of the native scalar basis, and several recently submitted alternatives [1]. The five incoming archives at the pinned snapshot were also inspected. In particular, five inference patterns, removal of first-class Q, canonical zero computation, and deletion of a root case from scalar evaluation have already been proposed [7, 8, 9]. None is claimed here as a first discovery.

The useful new change is to put the supplied equality in the *counter* of a counter-aware recursor. Its update can project that counter, which makes the same evaluation rule perform root equality composition. An update that applies any fixed term template to the counter makes it perform arbitrary congruence. Computation is the reflexive instance of the same rule.

This differs from the existing counter-congruence presentation, which retains separate root replacement and computation rules, and from its wrapped-counter refinement [2, 3]. It also differs from the scalar evaluation rule, whose first premise identifies a preceding *state* rather than an arbitrary counter [5, 9].

Complete standalone checker	Bytes	Tokens	AST nodes
Same counter language: computation + transport	3272	929	1063
Counter evaluation (this report)	**3004**	**854**	**987**
Scalar zero-normal, one-case evaluation	3061	884	1025
Incoming raw scalar CE5, unchanged	3046	853	993
Raw scalar CE5 with known root-case deletion	2999	848	986
Incoming canonical clause checker, unchanged	3305	946	1072

The first comparison keeps the term language, normalization, formula syntax, clause representation, validation, scoping, and public input contract fixed. The equality interface alone changes. Its reductions are 268 bytes (8.19%), 75 tokens (8.07%), and 76 AST nodes (7.15%). The other rows expose the actual tradeoffs rather than establishing dominance over every existing implementation. The new checker is smaller in all three coordinates than the scalar one-case zero-normal control, but not than the raw two-case control. Five-pattern calculi already existed; a smaller absolute rule count is not the claim.

All three costs count the complete checker, including normalization, scope validation, JSON input, and conclusion checking. No producer, translator, or project library is a trusted import. The historical compact checker family has a different certificate interface and much smaller source. Its 516-byte envelope remains below every readable checker in this table [6].

2 Numerical syntax and canonical substitution

Raw numerical terms have one sort and grammar

$$t ::= x \mid 0 \mid R(n, a; i, u.h).$$

Only the update h binds i, u. The first bound variable receives the old counter, the second the preceding state; passive numerical parameters are allowed. Define

$$S(a) := R(0, a; i, u.0).$$

The inherited intended computations are

$$R(0, a; i, u.h) = Sa, \tag{1}$$
$$R(Sn, a; i, u.h) = h[n/i, R(n, a; i, u.h)/u]. \tag{2}$$

This recursor and its elementary arithmetic compiler are prior work in the repository [2, 4]. We use two bound numerical variables in the update, unlike the later scalar iterator.

Definition 2.1 (Canonical terms)**.** Every child of a canonical term is canonical, and an R node whose counter is literally 0 has body literally 0. The smart constructor is

$$\rho(n, a, h) = \begin{cases} S(a), & n \text{ is literally } 0, \\ R(n, a; i, u.h), & \text{otherwise.} \end{cases}$$

Normalize a finite raw tree recursively by this constructor. Numerical substitution means ordinary capture-avoiding substitution followed by this normalization. All displayed formulas and rule instances use this convention.

Thus (1) is definitional. No successor unfolding is performed in normalization, and no test for *provable* equality with zero occurs. This is the existing canonical-zero convention, with two update binders instead of one [6]. It is part of the trusted syntax and is counted in the checker.

Normalization is idempotent and preserves scopes. It commutes with substitution in the form

$$\mathrm{nf}(t\sigma) = \mathrm{nf}\big(\mathrm{nf}(t)\ \mathrm{nf}(\sigma)\big).$$

For the R case, either the final counter is zero, when both sides erase the body, or it is not, when the structural induction hypotheses apply to all retained children. A body erased at an earlier stage cannot become relevant later: substitution preserves literal zero. The same argument gives composition of normalized substitutions and extends to the two formula constructors below. Semantic correctness under update bodies is treated separately in Section 8; it does not follow merely from syntactic idempotence.

A conventional first-order signature may replace every fixed finite body description by its corresponding function symbol, with its passive parameters as arguments. This is an externally generated signature. There is no function variable and no internal total evaluator for arbitrary primitive-recursive program codes.

3 The complete five-pattern calculus

Object formulas and retained clauses have the forms

$$A ::= (s = t) \mid D_{x,y}A, \qquad \langle A;\, y.B(y)\rangle,$$

where the clause variable y is absent from its antecedent A. Contexts are finite reusable lists of clauses. The ordinary readings are

$$D_{x,y}B := \exists x\big(B(x, 0) \to \forall y\, B(x, y)\big), \qquad \langle A;\, y.B\rangle := A \to \forall y\, B.$$

Let H be $0 = 0$. An ordinary local assumption A is represented by the constant clause $\langle H;\, v.A\rangle$, with v fresh and unused. Its use requires the reflexivity proof of H; there is no implicit hypothesis instruction. Write Γ, A as shorthand for adjoining this clause.

3.1 Reflexivity and counter evaluation

$$\frac{}{\Gamma \vdash a = a}\ (\text{Refl}), \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash R(Ss, a; i, u.h) = r}{\Gamma \vdash r = h[t/i, \rho(t, a, h)/u]}\ (\mathsf{CE}). \qquad \text{(Arithmetic)}$$

The substitutions in h are simultaneous and capture avoiding. The seed, body, and passive parameters are read from the second premise and kept fixed. In particular, occurrences inside a or h that happen to resemble s are not replaced. The first endpoint s is any term, including literal zero. The second endpoint t can also be zero, in which case $\rho(t, a, h) = Sa$.

3.2 Use, paired elimination, and paired introduction

$$\frac{\langle A;\, y.B\rangle \in \Gamma \qquad \Gamma \vdash A}{\Gamma \vdash B[t/y]}\ (\text{Use}),$$

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad \Gamma, \langle B(b, 0);\, y.B(b, y)\rangle \vdash F}{\Gamma \vdash F}\ (\text{DE}),$$

$$\frac{\Gamma, B(w, \eta) \vdash B(w, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}\ (\text{DI}), \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } w \text{ is literal } 0, \\ 0, & \text{otherwise.} \end{cases}$$

In DE, b is fresh for the outer context, conclusion, and passive matrix parameters. In DI, z is fresh for the outer context, witness, and passive matrix parameters. Bound variables are renamed to avoid capture. Use requires a well-scoped numerical target. All local clauses are discharged on exiting their indicated branch.

These are the entire primitive proof system. There is no additional induction, successor axiom, equality schema, structural proof node, logical connective, quantifier, or contradiction rule. Weakening, renaming, substitution, and cut are admissible operations proved below. The designated contradiction is the object equation $0 = S0$.

The label "five patterns" counts DI as one syntax-directed schema, although it has two specified modes. Its test is deliberately explicit. It is not a claim that the system has five interchangeable instructions under every possible rule-counting convention.

4 Equality and computation from one evaluation rule

Theorem 4.1 (Arithmetic reconstruction)**.** *Refl and CE alone derive successor computation, symmetry, transitivity, and crossed transport in every fixed numerical template:*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s) = r}{\Gamma \vdash r = F(t)}.$$

The constructions use no logical inference, numerical cases, or arithmetic induction.

Proof. For successor computation take the two reflexivities $n = n$ and $B = B$, where $B = R(Sn, a; i, u.h)$. CE concludes $B = h[n/i, \rho(n, a, h)/u]$, exactly (2).

For root composition suppose p proves $s = t$ and q proves $s = r$. Set

$$K_s := R(Ss, 0; i, u.i).$$

CE applied to q and $K_s = K_s$ gives $K_s = r$. CE applied to p and this result gives $r = t$. Denote this derived operation by

$$J(p, q) : (s = t), (s = r) \longmapsto (r = t).$$

Then $J(\mathrm{Refl}(s), p)$ is symmetry of p. Given $p : s = t$ and $q : t = v$, $J(q, \mathrm{Sym}(p))$ proves $s = v$.

For congruence choose the new binders fresh and put

$$B_F := R(Ss, 0; i, u.F(i)),$$

where the update ignores u. CE on two reflexivities gives $B_F = F(s)$. CE with first premise $p : s = t$ and that second premise gives $F(s) = F(t)$. Finally apply J to this congruence equation and the supplied $q : F(s) = r$. The result is $r = F(t)$. Every term substitution in this argument is a syntactic instance of CE. No congruence has been presumed in order to establish congruence. □

Proposition 4.2 (Explicit proof costs)**.** *Count every primitive inference, including a Refl leaf, as one node; count repeated subtrees repeatedly. Root composition has overhead three nodes, congruence has overhead four, and crossed transport has overhead seven. Each supplied premise tree occurs once.*

Proof. $J(p, q) = \mathsf{CE}(p, \mathsf{CE}(q, \mathrm{Refl}(K_s)))$ adds two CE nodes and one leaf. Congruence is $\mathsf{CE}(p, \mathsf{CE}(\mathrm{Refl}(s), \mathrm{Refl}(B_F)))$, adding four nodes. Crossed transport is $J(\mathrm{Cong}_F(p), q)$, adding $4 + 3 = 7$. Term labels contain copies of F and the endpoints, so these are inference-node bounds, not constant byte-size or runtime bounds. □

Theorem 4.3 (Exact equality-interface equivalence)**.** *Replace Refl and CE by successor computation and one-template crossed transport, leaving the canonical language and three logical patterns unchanged. Every sequent is derivable in one system exactly when it is derivable in the other.*

Proof. Theorem 4.1 gives one direction by effective replacement of each computation or transport node.

Conversely, successor computation with counter projection gives $K_s = s$. Applying crossed transport to two copies of this equation, with identity template, gives $s = s$. Symmetry and transitivity then follow from the root instances of crossed transport. For CE, equality $s = t$ gives

$$R(Ss, a; h) = R(St, a; h) = h[t/i, \rho(t, a, h)/u].$$

The first step is congruence in the fixed counter context; the second is successor computation. Reverse the second supplied premise $R(Ss, a; h) = r$ and compose. This proves the CE conclusion. The three logical rules are unchanged. Induction on finite proof trees therefore supplies both transformations, preserving end sequents. □

The use of ordinary equality in the converse is legitimate: it is derived from the reference interface before CE is simulated. The forward reconstruction never assumes it.

5 The logical and induction interface is complete

We give the metatheory needed to connect the five patterns to ordinary intuitionistic first-order logic. The clause and gate constructions are inherited [10, 7, 8]; their instantiation here is included to make the full theory explicit.

5.1 Recovering both introduction choices

Weakening and fresh-parameter renaming are already admissible by induction on primitive derivations: neither operation changes the literal-zero witness test. Local-assumption cut is admissible at this stage as well. Replace each Use of the constant clause $\langle H;\, v.A\rangle$ by the supplied proof of A, weakened into its local context and with eigenparameters renamed. This proof transformation does not require numerical substitution or full formula transport.

Consider an auxiliary calculus with two introductions: case introduction from $B(w,0)$ to $B(w,Sz)$ for every matrix, and progress introduction from $E(w,z)$ to $E(w,Sz)$ for literal equations only. The auxiliary calculus is stable under capture-avoiding numerical substitution.

The only case-introduction instance missing from primitive DI is an equation at witness zero. Let

$$Z := R(S0,0;i,u.0).$$

This is not literally zero, and successor computation proves $Z = 0$. Under $E(Z,0)$, equation transport gives $E(0,0)$. Apply the supplied case premise and transport $E(0,Sz)$ back to $E(Z,Sz)$. Primitive DI at witness Z now selects case mode and has exactly the desired conclusion $D_{x,y}E(x,y)$. Only equation transport was used.

For progress at any witness w, first introduce $D_{v,y}E(w,y)$ with unused v and witness zero. This is primitive progress mode. DE opens $\langle E(w,0);\, y.E(w,y)\rangle$. Under $E(w,0)$, Use supplies $E(w,Sz)$; the now-available case introduction at witness w gives the desired $D_{x,y}E(x,y)$. Closing DE completes the proof.

Thus the two calculi have identical consequence relations. Structural weakening, exchange, contraction, renaming, and numerical substitution follow by proof induction in the auxiliary calculus. For clarity, a general clause-replacement lemma has the premise $\Delta, A \vdash B(y)$, with y fresh for Δ, A; every Use of $\langle A;\, y.B\rangle$ at t is replaced by that derivation instantiated at t and local-assumption cut. These are derived operations. The single primitive witness test itself is *not* invariant under substitution: a variable witness can become zero. Admissibility is obtained through the auxiliary calculus, not by an invalid direct rule-for-rule argument.

5.2 Cases, induction, and formula transport

From $\Gamma \vdash A(0)$ and a fresh-parameter derivation $\Gamma \vdash A(Sz)$, case DI with an unused existential coordinate, followed by DE and Use, proves $\Gamma \vdash A(t)$. This is numerical case distinction for every object formula. Replacing case DI by equation progress yields contextual equation induction:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}. \tag{Eq-ind}$$

Only a literal equation is allowed as E; the retained context may contain arbitrary formulas or clauses.

Full object-formula transport follows by external structural induction on a formula template. Equation templates follow from numerical congruence and root composition. For $A(v) = D_{x,y}B(v,x,y)$, eliminate $A(s)$ at fresh b. Introduce $A(t)$ by case DI at b. Transport its local base $B(t,b,0)$ backwards to $B(s,b,0)$ using the smaller-template induction hypothesis, Use the source clause at Sz, and transport the result forwards to $B(t,b,Sz)$. Close DI and DE. This is induction on finite formula syntax in the metatheory, not induction on a compound arithmetical formula.

5.3 Explicit logical macros

Section 6 supplies a numerical selector C satisfying $C(0,a,b)=a$ and $C(Sn,a,b)=b$ by computation alone. For a formula A, define a gate G_vA on equation atoms by

$$G_v(s=t) := (C(v,0,s)=C(v,0,t)),$$

and commute it through D, freshening the binders. Then G_0A is provable and $G_{Sz}A$ is interderivable with A.

Define the padding $L(A) := D_{x,y}G_yA$, with x,y fresh. It is interderivable with A: introduce it by cases from A; conversely eliminate it and Use the provable zero gate at target 1. Every formula is a finite chain of D binders ending in an equation. Pad the shorter of A,B until their depths match. Rename corresponding binders identically and at their equation leaves define

$$M_v((s=t),(p=q)) := (C(v,s,p)=C(v,t,q)).$$

Carry matching D binders outside. Consequently $M_0(A,B)$ is interderivable with A, and $M_{Sz}(A,B)$ with B.

Write $Q_yA(y)$ as the object abbreviation $D_{x,y}A(y)$ with x unused. Choose a fixed term π with $\pi(1)=1$ and $\pi(2)=0$; for example ordinary recursion starting at zero and updating $u\mapsto C(u,1,0)$ has these two computation instances. Define

$$\begin{aligned}
\bot_* &:= (0=S0),\\
A\to_* B &:= Q_yM_y(A,B),\\
A\wedge_* B &:= Q_yG_yM_{\pi(y)}(A,B),\\
\exists_* x\,A(x) &:= D_{x,y}G_yA(x),\\
A\vee_* B &:= \exists_* v\,M_v(A,B),\\
\forall_* x\,A(x) &:= A(0)\wedge_* Q_xA(x).
\end{aligned}$$

All helper variables are fresh. These are finite definitions in Eq/D syntax; no extra object-formula constructor is added.

Verification of the intuitionistic rules. Implication introduction is case DI using the derivation from A to B. Elimination Uses the retained conditional closure at a successor. Conjunction introduction proves both alternatives of its selected matrix by numerical cases; elimination Uses targets 1 and 2 and the displayed computations of π. No range theorem for π is required: the general selector argument uses numerical cases on its value. Existential introduction is case DI at its witness, and elimination opens DE and Uses the zero gate at 1 to recover the witness formula. Disjunction introduction chooses numerical tag 0 or 1; elimination eliminates the existential tag and uses numerical cases and the two given branches. Universal introduction pairs the base instance with the conditional closure obtained from a fresh-parameter proof; elimination uses that base in the closure.

Finally, under $0=S0$, transport through $C(v,s,t)$ and its two computation equations yields every equation $s=t$. External induction on the number of D binders, using case introduction, yields every object formula. This verifies ex falso. These are precisely the usual intuitionistic introduction and elimination rules, with their usual discharge and eigenvariable restrictions. No excluded middle for quantified formulas is introduced.

Numerical coverage follows by cases on the macro formula $n=0\vee_*\exists_* m\,(n=Sm)$. Its zero and successor instances use the displayed witnesses. The designated contradiction supplies the separation formula $0=S0\to\bot$ in the ordinary realization.

5.4 Ordinary realization and both formula composites

Let T_R be ordinary intuitionistic first-order logic with actual equality over the recursor terms, the two computations (zero being canonical), coverage, separation $0 = S0 \to \bot$, and contextual literal-equation induction. Read D and clauses as specified in Section 3. Refl, CE, Use and DE are sound by equality and intuitionistic logic. Case DI is sound by coverage; progress DI is sound by equation induction under the retained base assumption.

Conversely, the preceding constructions realize every rule and axiom of T_R. The resulting translations are faithful in both directions. The ordinary reading of each logical macro is provably equivalent to the connective or quantifier it represents, by the same selector and gate arguments. For the other composite, externally induct on Eq/D syntax. In the D case, open its clause at a fresh witness and build the existential, implication and universal macros; conversely eliminate the existential macro, recover its conditional closure, and introduce D by cases. This proves native interderivability with its round trip, including formulas under binders and arbitrary retained assumptions.

Hence this is a reconstruction of the full first-order consequence relation, rather than only a sound translation or a coincidence of closed equational theorems.

6 The inherited arithmetic bridge in a short form

The following terms and compiler are inherited from the counter-aware recursor proposal [2, 4]. Their short proofs are a useful reason to use this language for counter evaluation.

Lemma 6.1 (Predecessor, selector, and ordinary recursion)**.** *Define*

$$\begin{aligned} P(n) &:= R(n, 0; i, u.i), \\ C(n, a, b) &:= P(R(n, a; i, u.Sb)), \\ \mathrm{Rec}(n, a; i, u.h) &:= P\big(R(n, a; i, v.S(h(i, P(v))))\big). \end{aligned}$$

Then the theory derives

$$\begin{aligned} P(0) &= 1, & P(Sn) &= n, \\ C(0, a, b) &= a, & C(Sn, a, b) &= b, \\ \mathrm{Rec}(0, a; h) &= a, & \mathrm{Rec}(Sn, a; h) &= h(n, \mathrm{Rec}(n, a; h)). \end{aligned}$$

These derivations use computation and equality only.

Proof. The P equations are the two computations with counter projection. The selector computations reduce the inner R to Sa or Sb, then apply $P(Sx) = x$. For recursion, put $T(n) = R(n, a; i, v.S(h(i, P(v))))$. Its base is Sa, while $T(Sn) = S(h(n, P(T(n))))$. Applying P proves the stated equations. □

Note $P(0) = 1$, not zero. The standard predecessor is instead

$$p(n) := C(n, 0, P(n)).$$

It satisfies $p(0) = 0$ and $p(Sn) = n$. Applying P to $Sx = Sy$ gives successor injectivity. The sign function $\mathrm{sg}(n) = C(n, 0, 1)$ gives $\mathrm{sg}(0) = 0$, $\mathrm{sg}(Sn) = 1$; hence $Sx = 0$ contradicts separation. Coverage also gives $n = 0 \vee n = S(p(n))$.

Every fixed primitive-recursive description now compiles into a finite term: zero, successor and projections have their evident translations; composition is substitution; primitive recursion is the displayed Rec construction. Unlike the scalar bootstrap, this compiler needs no pairing or finite-state predecessor construction. That advantage belongs to the inherited counter-aware basis, not to a new arithmetic claim here.

7 Why equation induction gives all quantifier-free induction

For completeness we give the constructive equality-reflection step. This avoids assuming that agreement with standard numerical evaluation already proves equivalence of theories on arbitrary models.

Using Rec, define addition $x + y$ by recursion on y with update S, and truncated subtraction $d(x, y)$ by recursion on y with update p. Thus

$$x + 0 = x, \quad x + Sy = S(x + y), \qquad d(x, 0) = x, \quad d(x, Sy) = p(d(x, y)).$$

Literal-equation induction proves $0 + y = y$ and $Sx + y = S(x + y)$. Inducting on y in the equation $p(d(Sx, y)) = d(x, y)$ proves

$$d(Sx, Sy) = d(x, y).$$

The base is $p(Sx) = x$; the step applies p to the induction equation. Induction on a therefore gives $d(Sa, a) = 1$.

Lemma 7.1 (Positive subtraction identifies a remainder)**.** *For all x, y,*

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{Remainder}$$

Proof. Induct on y in this single literal equation, keeping x fixed. At zero both selected values are x, using $0 + x = x$ and numerical cases on $\mathrm{sg}(x)$. For the step set $r = d(x, y)$ and $r' = p(r)$. If $r' = 0$, the new equation selects $x = x$. Otherwise coverage gives $r' = Sz$. Coverage on r, the predecessor equations, and separation then give $r = Sr'$. The old equation selects $y + r = x$. The new selected sum is

$$(Sy) + r' = S(y + r') = y + Sr' = y + r = x.$$

All branch distinctions are coverage. The sole induction hypothesis is the displayed equation, so no quantified or disjunctive induction has been used. □

If $d(Sa, b) = 1$, (Remainder) gives $b + 1 = Sa$. Since $b + 1 = Sb$, successor injectivity gives $a = b$. The converse follows from $d(Sa, a) = 1$. Define

$$K(n) := C(n, 1, p(n)), \qquad \epsilon(a, b) := \mathrm{sg}(K(d(Sa, b))).$$

Coverage and separation give $K(n) = 0 \leftrightarrow n = 1$ and $\mathrm{sg}(n) = 0 \leftrightarrow n = 0$. Therefore

$$\epsilon(a, b) = 0 \ \leftrightarrow\ a = b, \qquad \epsilon(a, b) = 0 \ \vee\ \epsilon(a, b) = 1.$$

These are internal whole-domain proofs, not checks on numerals.

For every fixed quantifier-free formula A, form a Boolean term b_A with zero meaning true:

$$\begin{aligned} b_{s=t} &= \epsilon(s, t), & b_{\bot} &= 1, \\ b_{A \wedge B} &= C(b_A, b_B, 1), & b_{A \vee B} &= C(b_A, 0, b_B), \\ b_{A \to B} &= C(b_A, b_B, 0). \end{aligned}$$

External induction on the finite formula, using the two Boolean cases, proves $b_A = 0 \leftrightarrow A$ and $b_A = 0 \vee b_A = 1$. In the implication case this uses the already established decidability of A, not excluded middle for arbitrary first-order formulas. An induction instance for $A(n)$ is now obtained from Eq-ind for $b_{A(n)} = 0$, under its unchanged retained context. This supplies exactly the quantifier-free induction of full first-order iPRA.

8 The two interpretations and their round trips

Here iPRA means intuitionistic first-order logic with equality, one function symbol for each fixed primitive-recursive description, its defining equations, successor injectivity and separation, and quantifier-free induction. The numerical domain of each interpretation below is the entire source domain. No quotient equality or proper cut is introduced.

8.1 From the recursor theory to iPRA

Translate the finite syntax of a fixed body $h(i, u, \vec{p})$ into a primitive-recursive function h^α. Introduce the corresponding fixed primitive-recursive description

$$r_h(0, a, \vec{p}) = Sa, \qquad r_h(Sn, a, \vec{p}) = h^\alpha(n, r_h(n, a, \vec{p}), \vec{p}).$$

Translate $R(n, a; h)$ by $r_h(n^\alpha, a^\alpha, \vec{p})$, and translate canonical zero nodes directly by successor. The translation $\alpha : T_R \to \mathsf{iPRA}$ leaves variables, actual equality, and ordinary logical constructors unchanged. Computations hold by definition; coverage is a quantifier-free induction consequence (equivalently, use the standard primitive-recursive predecessor); separation is an iPRA axiom; equation induction is an allowed induction instance.

8.2 From iPRA to the recursor theory

Translate every fixed primitive-recursive description by the compiler in Section 6. This defines $\beta : \mathsf{iPRA} \to T_R$ on the same domain with actual equality. All function defining equations hold by that compiler; successor properties were proved using P and sg; quantifier-free induction is Section 7. Extend identically through ordinary first-order logic. Thus both directions interpret the *full* first-order theories.

8.3 Body extensionality and substitution coherence

Suppose two fixed bodies are pointwise equal under a retained context, and their seeds are equal. Equation induction on a fresh counter proves the equality of the corresponding recursors. The base is successor congruence on the seeds. At the step, the two successor computations expose the bodies; replace the previous state using the induction equation, then use the pointwise body equality at the counter and that state. All passive parameters remain fixed.

This lemma justifies semantic substitution when two differently written body descriptions produce different function symbols. Literal identity of those symbols is not assumed. It also proves correctness of recursive zero normalization inside arbitrary enclosing recursors: induct externally on syntax, use body extensionality for the body child, and use the zero equation when a normalized counter is zero. The target theory already has equation induction at this stage; there is no circular use of a desired transport theorem.

8.4 Both composites are internally the identity

For a fixed R term, use external induction on its finite syntax. The induction hypotheses identify the children and the translated body. The original R trajectory and the compiled trajectory of r_h share initial value Sa and update $h(n, u)$. One literal-equation induction on a fresh counter proves their equality at every counter, under all passive parameters. Substitution at the original counter gives

$$T_R \vdash t^{\alpha\beta} = t.$$

For a fixed primitive-recursive description, induct externally on that description. Zero, successor, projection and composition are immediate. In a recursion case the original and translated functions have equal bases and steps by the description induction hypotheses; one iPRA equation induction proves equality everywhere. Hence

$$\mathsf{iPRA} \vdash t^{\beta\alpha} = t.$$

The superscripts here indicate successive application in written order. Extending externally over arbitrary first-order formulas gives provable equivalence with each round trip. The native/ordinary formula composites were established in Section 5.

Theorem 8.1 (Main result)**.** *The five-pattern calculus $\mathcal{K}_R$ is bi-interpretable and equiconsistent with full first-order iPRA. Both numerical interpretation domains are $x = x$, equality is actual equality, and the graph of each internally definable numerical isomorphism is $x = y$. These translations preserve the full consequence relation, including open sequents and formulas with unbounded quantifiers.*

Proof. Compose the faithful native/ordinary translations with α and β. Their formula composites are provably equivalent to the original formulas and their numerical composites are the displayed termwise identities. The graph $x = y$ is therefore an internal total bijection preserving all interpreted operations and relations. The native contradiction is $0 = S0$, whose ordinary reading implies and is implied by falsity using separation and explosion. Inconsistency thus transfers in both directions, proving equiconsistency as a consequence of the stronger bi-interpretability theorem. □

9 Executable artifacts and verification boundary

9.1 The public certificate contract

The standalone checker reads one JSON object with exactly the keys `goal` and `proof`. The goal is closed. Exit status zero and the line `ACCEPT` constitute acceptance; every exception causes rejection. No Python expression in an input is evaluated. As with the comparison programs, a concrete Python run has recursion and memory limits and can reject an oversized valid certificate. Completeness is a statement about the finite formal calculus and the checker under unbounded resources, not a promise that a fixed machine can process every finite proof.

Terms use the integer 0, negative free-variable levels, positive bound indices, and three-element arrays $[n, a, h]$. Inside an R body, index 1 is the state and index 2 the counter. A D body also binds two variables, with y at index 1 and x at index 2. Equations are $[s, t]$ and paired formulas are $[B]$. Tree arities therefore distinguish every syntactic constructor. Booleans are rejected as integer variables; all scopes and canonical bodies are validated.

An opening traversal replaces one bound slot and removes it. Since all external variables are negative levels, opening a body with an externally scoped term introduces no free positive index to capture. The traversal tracks the two local binders of every R or D body and reapplies the

canonical constructor. CE opens the state slot and then the counter slot, implementing simultaneous substitution. This order and the normalization of $\rho(t, a, h)$ are essential.

The proof instructions are `r` (reflexivity), `v` (counter evaluation), `u` (clause use), `e` (DE), and `i` (DI). The latter two introduce one fresh negative level and check their branch conclusion. A freshness escape is rejected by the scope and branch checks. The full source is Appendix B.

9.2 Effective proof conversion

The archive includes the computation-plus-transport control on the same canonical terms and an untrusted translator to counter-evaluation proofs. A computation node becomes CE on two reflexivities. A transport node becomes the congruence-plus-root construction of Proposition 4.2. Use, DE and DI copy their data and translate their premise trees recursively. End formulas are preserved literally. This is a uniform construction on finite certificates, not a search through semantic truth values. The output is independently checked by the standalone target checker.

9.3 What was checked

The independent validation suite exercises symbolic root composition, successor computation, arbitrary-template congruence and crossed transport, including templates with nested recursors and passive parameters. It checks the exact +3, +4, and +7 node overheads. It also exercises logical branches, an induction proof that uses its local induction hypothesis, and rejection after that use is removed. Negative cases include a false equality, mismatched CE counters, noncanonical zero bodies, Boolean and out-of-scope variables, malformed formulas, and eigenvariable or witness escapes.

Finite numerical checks separately test the recursor, selector, predecessor and primitive-recursion compiler. They are a supplement to the symbolic proof arguments, not a substitute for them. The reproduction script writes the exact test counts and machine-readable results into `evidence/`. The recorded run includes 37 positive and 15 negative public certificates, each run normally and with Python optimization (104 process checks); 44 additional open arithmetic and substitution checks; 29 nested-template congruence/transport pairs; and 570 numerical/compiler checks, all with the expected outcomes. Both proof translations additionally passed 37 closed and four open round trips, with all 15 invalid source certificates rejected. The comparison-control suite checks the existing root-case deletion and the scalar zero-normal variant separately.

The new equivalence and bi-interpretation proofs are written mathematics. The included checkers and finite tests are not a new Lean or Rocq verification. The repository's existing Lean results certify its own native scalar calculus; they do not automatically certify this new primitive rule or its Python implementation.

9.4 Reproducible source metrics

Metrics use CPython 3.12.14, remove the module docstring, apply `ast.unparse`, and append one newline. Significant tokens exclude comments, formatting tokens, and the end marker; AST nodes are counted by `ast.walk`. Gzip-9 with timestamp zero is recorded only as supplementary data. The exact source programs and the measurement script are included. The source-size comparisons in Section 1 do not omit validation, parsing, or error handling.

10 Further research questions

1. Can the counter-evaluation interface be deeply embedded in the repository's Lean development, with a checked equivalence to its existing native calculus? The arithmetic reconstruction is small; the main added work is the two-binder syntax and canonicalization.
2. Can one obtain a uniform raw-syntax counter rule with comparable checker size, accounting explicitly for the zero computation rather than putting it into canonical substitution?
3. Can the two counter-body instantiations be represented with a simpler complete scope discipline, improving the transparent checker without moving validation or normalization outside the trusted code?
4. Can an explicit proof-complexity comparison between counter CE and scalar state evaluation quantify certificate lengths and label sizes as well as primitive inference nodes?
5. The incoming raw-clause report's restricted successor-rigidity principle yields zero-equation induction but leaves a faithful equality/predecessor bootstrap unresolved [8]. Does that strictly narrower arithmetic principle suffice? The present report does not solve that gap or borrow its unproved conclusion.
6. Which robust complexity measure can compare genuinely different proof interfaces? The current byte, token, AST, constructor, and proof-expansion coordinates disagree. A global simplicity claim would need a specified coding model and a corresponding lower-bound theorem.

A Novelty audit and attribution

The audit used the pinned repository index, all five incoming archive reports or claim files, and the full relevant specifications for counter congruence, its restricted/wrapped refinement, computational equality, canonical recursors, initializer equality, and the raw and canonical clause presentations. This is a scoped comparison, not a claim to have read every archived file or established literature-wide priority.

Ingredient	Attribution / distinction
Counter-aware successor-seeded R, P, C, and Rec	Existing successor-recursion basis; recorded in counter-congruence and scalar-successor reports.
Zero-body canonicalization	Canonical-recursors proposal; two numerical binders here.
Eq/D clauses and five-pattern count	Incoming canonical-clausal and raw-clausal proposals.
Witness-directed single DI	Existing witness-rule proposal.
Scalar root-case deletion	Incoming equality-from-iteration report; not rediscovered here.
Refl plus uniform counter CE	Proposed interface of this report; root composition and congruence follow from CE itself, without separate root replacement or computation axioms.

The included baseline checker files are retained under the repository's MIT No Attribution license. Mathematical attribution is retained independently of that license. Source references and SHA-256 checksums of the delivered artifacts are supplied for reproducibility.

B Complete standalone checker

The following listing is the entire trusted implementation. The separate producers, translators, comparisons, and tests are not imported.

```
"""Counter-aware canonical recursor with one evaluation scheme.

Terms are 0, negative free levels, positive bound indices, and [n,a,h].
A recursor body binds state u at index 1 and counter i at index 2.
R(0,a;h) is canonical only with h=0; successor(a) = [0,a,0].
Formulas are [s,t] (equality) and [B] (paired binder D).
Input is {'goal': a closed formula, 'proof': a finite proof tree}.
"""
import json
import sys


def fail():
    raise ValueError('invalid certificate')


def rec(n, a, h):
    return [n, a, 0 if n == 0 else h]


def successor(a):
    return [0, a, 0]


def term(a, scope, depth=0):
    if type(a) is int:
        return -scope <= a <= depth
    return (type(a) is list and len(a) == 3
            and (a[0] != 0 or a[2] == 0)
            and all(term(t, scope, depth + 2 * (i == 2))
                    for i, t in enumerate(a)))


def formula(a, scope, depth=0):
    return (type(a) is list
            and ((len(a) == 2 and all(term(t, scope, depth) for t in a))
                 or (len(a) == 1 and formula(a[0], scope, depth + 2))))


def opening(a, value, depth=0):
    if type(a) is int:
        return value if a == depth + 1 else a - (a > depth + 1)
    size = len(a)
    result = [opening(t, value, depth + 2 * (size != 2 and i == size - 1))
              for i, t in enumerate(a)]
    return rec(*result) if size == 3 else result


def derive(proof, context, scope):
    if type(proof) is not list or not proof or type(proof[0]) is not str:
        fail()
    tag = proof[0]
    if tag == 'u' and len(proof) == 4:
        index, witness, premise = proof[1:]
        if (type(index) is not int or not 0 <= index < len(context)
                or not term(witness, scope)):
            fail()
        antecedent, body = context[index]
        if derive(premise, context, scope) != antecedent:
            fail()
        return opening(body, witness)
    if tag == 'r' and len(proof) == 2:
```

```
        a = proof[1]
        if not term(a, scope):
            fail()
        return [a, a]
    if tag == 'v' and len(proof) == 3:
        s, t = derive(proof[1], context, scope)
        b, r = derive(proof[2], context, scope)
        c, a, h = b
        if c != successor(s):
            fail()
        return [r, opening(opening(h, rec(t, a, h)), t)]
    if tag in ('e', 'i') and len(proof) == 4:
        goal, data, branch = proof[1:]
        if not formula(goal, scope):
            fail()
        fresh = -scope - 1
        if tag == 'e':
            paired = derive(data, context, scope)
            if len(paired) != 1:
                fail()
            body = opening(paired[0], fresh, 1)
            clause = (opening(body, 0), body)
            target = goal
        else:
            if len(goal) != 1 or not term(data, scope):
                fail()
            body = opening(goal[0], data, 1)
            eta = fresh if len(goal[0]) == 2 and data == 0 else 0
            clause = ([0, 0], opening(body, eta))
            target = opening(body, successor(fresh))
        if derive(branch, context + [clause], scope + 1) != target:
            fail()
        return goal
    fail()


def verify(data):
    if type(data) is not dict or set(data) != {'goal', 'proof'}:
        fail()
    if not formula(data['goal'], 0):
        fail()
    if derive(data['proof'], [], 0) != data['goal']:
        fail()
    return True


if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

References

[1] ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic*. README at commit `831081033253cca2115196cf86bb99c77ed2aeae`. All repository paths below are relative to `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/` at this snapshot. https://github.com/VladimirReshetnikov/ProveIt/tree/831081033253cca2115196cf86bb99c77ed2aeae/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt contributors. *Counter congruence recovers unrestricted equality transport*. `Research/counter_congruence/theory/counter_theorem.tex`.

[3] ProveIt contributors. *A narrower congruence rule and its exact boundary*. `Research/witness_rule/theory/restricted_congruence.tex`.

[4] ProveIt contributors. *Scalar successor*. `Research/scalar_successor/report.tex`, especially the attribution of the earlier counter-aware successor-recursion basis.

[5] ProveIt contributors. *Computational equality*. `Research/computational_equality/` and `Lean/ConciseIPRA/NativeEval.lean`.

[6] ProveIt contributors. *Canonical recursors*. `Research/canonical_recursors/README.md` and `theory/canonical_R.tex` within that package.

[7] ProveIt contributors. *Clausal canonical presentation*. `docs/incoming/ipra_clausal_canonical.zip`.

[8] ProveIt contributors. *Five rules for iPRA with ordinary substitution*. `docs/incoming/ipra_raw_clausal_2026-11-11.zip`, especially the rule specification, finite-seed obstruction, and restricted successor-rigidity discussion.

[9] ProveIt contributors. *Equality from iteration*. `docs/incoming/ipra_equality_from_iteration_2026-11-11.zip`.

[10] ProveIt contributors. *Complementary introductions with one local assumption* and *From literal-equation induction to full intuitionistic PRA*. `Research/certified_zero/reference/inherited_choice_basis.tex` and `equality_reflection.tex`.

[11] ProveIt contributors. *Equality in the initializer*. `Research/initializer_equality/sections/seed_rule.tex`.